Architectural Specification for Autonomous Skill Harvesting and Protocol Expansion in the Stela Platform

1. Executive Summary

The evolution of agentic artificial intelligence has reached a critical inflection point where the primary constraint on performance is no longer raw model capability, but rather the efficient management of context and the persistence of procedural knowledge. In the specific operational environment of the Stela platform, a provenance-first, file-based repository system, the mechanism for capturing "lessons learned" remains a manual, operator-dependent process. This reliance on human or agent initiative to log skills creates a systemic vulnerability known as "Skill Rot," where valuable execution patterns are lost to the ephemeral nature of context windows immediately upon task completion.

This report presents a comprehensive architectural specification for the expansion of ops/lib/skill/skill_lib.sh from a rudimentary logging utility into a sophisticated **Autonomous Skill Harvesting Engine**. By integrating this engine directly into the mandatory closeout phase of the Dispatch Packet (DP) lifecycle defined in TASK.md, the system transforms from a passive repository into a self-improving organism. The proposed architecture leverages heuristic analysis of git artifacts, structured logging principles, and "Drift Prevention" protocols to automatically nominate high-fidelity skill candidates. This shifts the role of the AI Contractor from a mere executor of tasks to a curator of its own expertise, ensuring that every commit contributes to a durable, modular library of S-LEARN capabilities. The following analysis draws upon internal repository canon—specifically TRUTH.md, AGENTS.md, and SKILL.md—and synthesizes these with external industry standards regarding Agent Skills, structured observability, and autonomous frameworks to provide a definitive roadmap for implementation.

---

2. The Epistemological Crisis in Stateless Agentic Systems

To understand the necessity of expanding ops/lib/skill/skill_lib.sh, one must first grapple with the fundamental epistemological deficiency inherent in modern Large Language Model (LLM) architectures: the "Context Problem".[1] Unlike human engineers who accumulate tacit knowledge over years of practice—internalizing muscle memory for syntax, architectural patterns, and debugging heuristics—agents are born tabula rasa at the start of every session. They possess vast general knowledge but lack the specific, procedural intimacy with the local

repository that characterizes a "senior" developer.

2.1 The Context-Skill Dichotomy

In the Stela architecture, as defined by docs/CONTEXT.md [2], "Context" is the precise set of artifacts required to rehydrate an agent. However, loading the entire repository into the context window is computationally prohibitive, costly, and degrades reasoning performance due to the "lost in the middle" phenomenon. This necessitates a strict architectural separation between **Context** and **Skill**.

Feature	Context (The "Where")	Skill (The "How")
Definition	State-specific facts about the current moment.	State-agnostic procedural knowledge valid across time.
Example	"The bug is in line 42 of auth.py."	"How to safely patch auth.py using the Zenith pattern."
Lifespan	Ephemeral (Valid only for the current DP).	Durable (Valid until the platform architecture changes).
Storage	TASK.md, OPEN prompt.	docs/library/skills/, ops/lib/skill/.
Harvesting Goal	Discard after use.	Extract, generalize, and preserve.

The "Context Hazard" explicitly identified in Stela's SKILL.md [2]—"Skills must never be added to ops/lib/manifests/CONTEXT.md"—is a protective measure to prevent token bloat. It enforces a model of "Progressive Disclosure" [3], where agents must "check out" knowledge like a library book. The harvesting process, therefore, is the mechanism of writing these books based on the agent's field experiences.

2.2 The Risk of Skill Rot in Manual Systems

Currently, the Stela repository relies on a manual invocation of ops/lib/skill/skill_lib.sh. This assumes that after an exhausting debugging session, the agent (or operator) will have the cognitive surplus to step back, abstract the lesson, and format it into a valid S-LEARN entry.

Industry analysis of developer behavior and agent reliability suggests this assumption is flawed. When "Closeout" is viewed as a checklist of administrative chores, knowledge capture becomes the first casualty. The result is a repository that grows in code size but stagnates in "intelligence," as the same problems are solved de novo by successive agents who lack access to the prior agent's solution path. This phenomenon, termed "Skill Rot," creates a ceiling on the platform's complexity, as agents spend proportionally more time rediscovering basic patterns rather than advancing the architecture.

---

3. Forensic Audit of the Current Stela Infrastructure

A detailed examination of the existing artifacts in the Stela repository reveals a solid foundation that is underutilized due to tooling limitations. The dump-full-main-b818b6193.txt [2] provides the primary evidence for this audit.

3.1 Analysis of SKILL.md (The Governance Layer)

The SKILL.md file serves as the "Promotion Template" for creating new skills. It defines a rigorous "Promotion Packet" structure that includes:

- **Drift Preventers:** Stop conditions and anti-hallucination rules.
- **Definition of Done:** Requirements for index updates and SoP entries.
- **Verification:** Commands to prove the skill works.

While the *schema* is robust, the *method of population* is fragile. The file acts as a log, expecting users to append candidate entries manually. The "Candidate Log" section at the bottom of the file is intended to be an append-only ledger, but without automated enforcement, it risks becoming a dumping ground for low-quality, unstructured text.

3.2 Analysis of ops/lib/skill/skill_lib.sh (The Tooling Layer)

The current script is a rudimentary bash utility designed for basic string appending. Its limitations are severe when viewed against the requirement to "actually harvest genuine learned skills."

3.2.1 The Single-Line Constraint

The script accepts three positional arguments: name, context, and solution. Crucially, it includes validation logic that rejects multi-line inputs:

Bash

```
if [[ "$name" == *$'\n'* |

| "$context" == *$'\n'* |
| "$solution" == *$'\n'* ]]; then
  echo "ERROR: inputs must be single-line values" >&2
  exit 1
fi
```

This constraint is antithetical to the nature of technical expertise. A "genuine learned skill"—such as a complex refactoring pattern or a security hardening protocol—cannot be adequately described in a single line of text. This forces the agent to trivialize the skill (e.g., "Fixed a bug") just to pass the script's validation, stripping away the nuance that makes the skill valuable.

3.2.2 Lack of Context Awareness

The script operates in a vacuum. It has no awareness of the TASK.md file, the active Dispatch Packet (DP), or the git status. It relies entirely on the user to provide the context. This means if an agent fixes a critical issue in ops/bin/dump but types "Fixed documentation" into the context field, the script accepts it without validation. There is no automated link between the *work performed* and the *skill recorded*, creating a provenance gap that undermines the "Truth" of the repository.

3.2.3 The "Ghost Artifact" Problem

The script generates a text block called a "Promotion Packet" and appends it to SKILL.md, but it *does not create the skill file itself*. It leaves the "Definition of Done" unfulfilled. The operator or agent must manually copy-paste this text into a new file in docs/library/skills/. This friction increases the likelihood of "Ghost Artifacts"—skills that are listed in the log but do not exist as executable documentation, leading to failures in the tools/lint_library.sh registry guard.[2]

3.3 Analysis of TASK.md (The Workflow Layer)

The TASK.md template serves as the living work surface for the thread. Currently, it includes a generic instruction in the "Receipt" section:

> "Run ops/lib/skill/skill_lib.sh at least once to capture a lesson learned from the DP work."

This instruction is passive. It does not gate the completion of the task. An agent can check the box without actually running the script, or run it with low-quality inputs, and the "Definition of Done" for the DP will technically be met. To "actually harvest" skills, this step must be elevated from a suggestion to a mechanical gate, enforced by the same rigor applied to repo-gates.[2]

---

4. Theoretical Framework for Automated Harvesting

To transition from manual logging to automated harvesting, we must adopt a theoretical framework that treats the repository activity as a signal to be decoded. This approach leverages concepts from "Structured Observability" and "Agentic Design Patterns."

4.1 The Harvesting Heuristic: Signal from Noise

The core challenge in automation is distinguishing between routine maintenance (noise) and novel patterns (signal). Not every commit represents a new skill. Correcting a typo in README.md is not a skill; refactoring the README to align with a new "Tone Lane" policy *is* a skill.

The harvesting engine must apply a heuristic filter based on the "Levels of Autonomy" [5] and the "epistemic value" of the change.

- **High-Value Signal:** Changes to ops/, tools/, or governance files (TRUTH.md, AGENTS.md) usually indicate a shift in *how* the system operates. These are prime candidates for harvesting.
- **Medium-Value Signal:** Changes to projects/ that introduce new architectural patterns (e.g., a new way of handling database connections in FastAPI [6]) are valuable if they are intended to be reused.
- **Low-Value Signal:** Minor text edits, dependency bumps, or one-off bug fixes in content files.

The expanded skill_lib.sh must be able to read the git diff --stat and git diff --name-only to classify the work into these categories.

4.2 Structured Knowledge Acquisition

Best practices in "Agent Skills" [1] emphasize that skills are not just text; they are structured data. A valid skill typically contains:

1. **Metadata:** ID, Name, Scope.
2. **Invocation:** The "Trigger" condition.
3. **Procedure:** The executable steps.
4. **Verification:** The test condition.

The "Harvesting" process is effectively a **Structure Extraction** task. It takes the unstructured data of the work log and the semi-structured data of the diff, and maps them into the rigid schema of the S-LEARN template. This aligns with the "Structured Logging" philosophy [7], where logs are emitted as JSON objects (or distinct key-value pairs) rather than free text, enabling downstream analysis. By enforcing this structure at the point of capture, we ensure the skill is machine-readable by future agents.

4.3 The "Contractor" as Curator

In the Stela AGENTS.md [2], the "Contractor" role is defined as: "Executes specific logic tasks and drafts implementation details within a defined scope." This role definition must be conceptually expanded. The Contractor is not just a laborer; they are a field researcher. Their primary output is code, but their secondary output is *methodology*. By automating the nomination process, we empower the Contractor to fulfill this secondary mandate without distracting from their primary objective. The script becomes their "Field Notebook," automatically organizing their observations into a format that the "Integrator" (the governance layer) can validate.

---

5. Architectural Blueprint: The Skill Harvesting Engine

We propose a radical restructuring of ops/lib/skill/skill_lib.sh. It will cease to be a simple appending utility and will become a state-aware orchestration tool with three distinct operational modes: **Harvest**, **Draft**, and **Promote**.

5.1 System Architecture Diagram (Conceptual)

The following table outlines the flow of data through the proposed Harvesting Engine.

Stage	Input Source	Processing Logic	Output Artifact
1. Sensing	TASK.md (Objective), git diff (Scope)	Heuristic Analysis (Category detection)	Raw Context Data
2. Drafting	Raw Context Data	Template Injection (Pre-filling fields)	storage/handoff/SKILL_CANDIDATE.md
3. Refinement	Agent/User Input	Human-in-the-loop Editing	Refined Candidate File
4. Promotion	Refined Candidate	Validation (Linting), ID Generation	docs/library/skills/S-LEARN-XX.md
5. Registration	Promoted File	Registry Management	Updates to SKILL.md & INDEX.md

5.2 The Input Pipeline (Sensing Phase)

The script must first "orient" itself within the current work session.

- **Task Parsing:** The script parses TASK.md to extract the current DP-OPS-ID and the stated "Objective." This provides the semantic "Why" of the skill.
- **Diff Analysis:** The script runs git diff --name-only (against HEAD~1 or main) to identify the "What." It applies regex filters to determine the domain:
  - ^ops/.* -> Domain: Platform Operations.
  - ^docs/.* -> Domain: Documentation Standards.
  - ^projects/.* -> Domain: Project Payload.
- **Verification Check:** It scans storage/handoff/DP-OPS--RESULTS.md (if it exists) to see if verification steps passed. If "Verification: NOT RUN" is found, the harvest is flagged as "Unverified."

5.3 The Drafting Mechanism (Nomination Phase)

Instead of forcing the user to invent a skill from scratch, the script generates a **Skill Candidate File** in the storage/handoff/ directory. This file is pre-populated with data derived from the sensing phase.

- **Auto-Naming:** If the DP title is "Refactor Auth," the candidate name defaults to "Auth Refactoring Pattern."
- **Auto-Scoping:** If the diff only touches projects/, the scope is set to "Production Payload Only."
- **Auto-Drift Prevention:** The script injects standard drift preventers based on the file types touched. (e.g., if .sh files are modified, it adds: "Check executable permissions").

This "Drafting" step is crucial. It changes the user experience from "Writer's Block" to "Editor's Review." It is easier to correct a draft than to write one.

5.4 The Promotion Mechanism (Canonization Phase)

Once the draft is reviewed, the promote command finalizes it.

- **ID Allocation:** The script scans docs/library/skills/ and docs/library/INDEX.md to find the highest S-LEARN-XX ID and increments it. This prevents collision and ensures an unbroken sequence.
- **File Creation:** It effectively mvs the candidate to the docs/library/skills/ directory, renaming it with the canonical ID.
- **Registry Update:** It appends the new skill to docs/library/INDEX.md in the correct "Skills" section, ensuring the tools/lint_library.sh checks will pass.
- **Log Entry:** It appends a summary to SKILL.md as a historical record.

---

6. Implementation Specification: ops/lib/skill/skill_lib.sh v2

This section provides the specific technical requirements for rewriting the script. The new script must be robust, error-tolerant, and capable of handling multi-line text blocks.

6.1 Functional Requirements

1. **Flag-Based Interface:** The script must support --harvest, --promote, and --help flags.
2. **State Awareness:** It must fail gracefully if TASK.md is missing or if not run from the repo root.
3. **Heredoc Support:** It must use temporary files or heredocs to manage multi-line content, bypassing the limitation of command-line arguments.
4. **Atomic Operations:** Updates to INDEX.md and SKILL.md must happen only if the file creation succeeds.

6.2 Detailed Logic Flow for --harvest

Bash

```
# Pseudocode logic for harvest mode
function harvest_skill() {
    # 1. Read TASK.md
    DP_ID=$(grep "^# DP-OPS" TASK.md | head -1 | awk '{print $2}' | tr -d ':')
    OBJECTIVE=$(grep -A 1 "## II) SCOPE" TASK.md | tail -1)

    # 2. Get Changed Files
    FILES=$(git diff --name-only HEAD~1)

    # 3. Determine Category
    if]; then CATEGORY="Platform Ops"; fi

    # 4. Create Template
    cat <<EOF > storage/handoff/SKILL_CANDIDATE.md
# Skill Candidate (Auto-Generated from $DP_ID)
## Metadata
- Name:
- Scope: $CATEGORY
- Source: $DP_ID

## Invocation Guidance
```

```
Use this skill when attempting to achieve: "$OBJECTIVE"

## Solution
(The agent must fill this section with the specific pattern derived from the diff)

## Drift Preventers
- [ ] Stop if context does not match $CATEGORY
EOF
}
```

6.3 Detailed Logic Flow for --promote

```
Bash
```

```
# Pseudocode logic for promote mode
function promote_skill() {
  CANDIDATE=$1

  # 1. Validate Candidate
  if grep -q "\+" | sort -n | tail -1)
  NEXT_ID=$(printf "S-LEARN-%02d" $((LAST_ID + 1)))

  # 3. Publish
  cp "$CANDIDATE" "docs/library/skills/$NEXT_ID.md"

  # 4. Register
  echo "skill-${NEXT_ID,,} | Skill: $(grep "Name:" "$CANDIDATE") | docs/library/skills/$NEXT_ID.md" >>
docs/library/INDEX.md
}
```

---

7. Integration with TASK.md and Governance Protocols

The technology (the script) is only as effective as the protocol (the instructions). The "Missing Link" in the current system is the integration of this tool into the mandatory workflow.

7.1 The Revised Closeout Protocol

We must rewrite the "Closeout" section of the TASK.md template. This section is the "checklist" that the agent must complete before declaring the task done. By inserting the harvesting step here, we make it unavoidable.

Proposed TASK.md Section 3 Update:

3) CLOSEOUT (MANDATORY)

A. Skill Harvesting (The Knowledge Gate)

- **Run:** ops/lib/skill/skill_lib.sh --harvest
- **Action:** Review the generated storage/handoff/SKILL_CANDIDATE.md.
  - *Option A (New Skill):* Refine the draft (fill in Solution/Verification) and run --promote.
  - *Option B (No Skill):* If the work was routine/trivial, delete the draft and log "No new skill harvested" in the Work Log.
- **Constraint:** You must explicitly decide. Do not skip this step.

B. Worker Closeout Duties

- [ ] Produce/Update: storage/handoff/DP-OPS--RESULTS.md
- [ ] Update SoP.md if canon changed.

7.2 The "Curator" Stance in AGENTS.md

We must also update AGENTS.md to reflect this new responsibility. The "Contractor" is no longer just a coder; they are a "Knowledge Engineer."

- **Add to Contractor Responsibilities:** "Identify and harvest reusable patterns from execution logs. Ensure new skills are generalized and free of specific project context."

7.3 Governance and Quality Control

Automated harvesting creates a risk of "Hallucination Amplification," where an agent invents a fake pattern and encodes it as a skill. To mitigate this:

1. **The Verification Gate:** The --promote function should check for the existence of storage/handoff/DP-OPS--RESULTS.md. If the DP was not verified (i.e., tests were not run), the script should refuse to promote the skill.
2. **The Library Guard:** We must update tools/lint_library.sh to validate the *internal structure* of the new skill files, ensuring they adhere to the SKILL.md schema (e.g., must have a "Drift Preventers" header).
3. **Operator Review:** The final PR review by the human Operator serves as the ultimate sanity check. The Operator looks at the new S-LEARN file and asks: "Is this actually true?"

---

8. Strategic Implications: The Self-Optimizing Repository

Implementing this architecture moves Stela towards a Level 3 Autonomy framework [5], where the system actively collaborates in its own improvement.

8.1 Solving the "Blank Page" Problem

The primary friction in knowledge management is the "Blank Page Problem." Engineers hate writing documentation from scratch. By using the --harvest mode to pre-fill 60% of the skill file (Metadata, Scope, Context, Preventers), we reduce the activation energy required to capture the remaining 40% (the core logic). This dramatically increases the volume and velocity of skill capture.

8.2 Building a Structured Knowledge Graph

Over time, the collection of S-LEARN files creates a structured knowledge graph of the platform's capabilities. Because these files are machine-readable and standardized, they can be easily ingested by vector databases for RAG (Retrieval Augmented Generation) workflows.[8] Future agents can query this database to find "How to add a new CLI tool" and retrieve the exact, verified pattern harvested from a previous successful DP.

8.3 Security and Privacy Considerations

A critical aspect of automated harvesting is preventing the leakage of secrets or PII (Personally Identifiable Information) into the skill library.[6]

- **Redaction Protocol:** The --harvest logic must include a filter (using sed or regex) to strip out potential secrets (e.g., strings resembling API keys) or specific user data (e.g., email addresses) from the diff before populating the candidate file.
- **Scope Isolation:** By rigorously separating ops/ skills from projects/ skills, we prevent project-specific business logic (which might be sensitive) from leaking into the platform's general operating procedures.

---

9. Conclusion and Recommendations

The expansion of ops/lib/skill/skill_lib.sh is not merely a scripting task; it is a governance upgrade. By automating the nomination of skill candidates, Stela bridges the gap between the ephemeral nature of agentic work and the enduring need for platform expertise. The proposed "Harvest-Draft-Promote" workflow, enforced by the TASK.md closeout gate, turns every Dispatch Packet into a potential lesson, ensuring that the platform gets smarter with every commit.

9.1 Immediate Recommendations

1. **Refactor ops/lib/skill/skill_lib.sh:** Implement the --harvest and --promote modes immediately, prioritizing the interaction with git diff and TASK.md.
2. **Update Templates:** Modify TASK.md to include the mandatory "Skill Harvesting" section in the Closeout block.
3. **Backfill Harvest:** Perform a manual harvest of the last five completed DPs in SoP.md to seed the library with initial high-quality skills and test the heuristic logic.
4. **Strengthen Gates:** Update tools/lint_library.sh to enforce the schema of the new skill files, ensuring that the "Curator" agent maintains high standards.

This architectural shift ensures that Stela remains a cutting-edge example of a "Self-Documenting, Provenance-Forward" AI platform, capable of sustaining complex operations through the autonomous accumulation of genuine learned skills.

---

10. Operational Playbook: The Harvest Workflow

The following table summarizes the new operational cadence for the Contractor agent.

Phase	Action	Tool / Command	Outcome
1. Execution	Perform the work defined in the DP.	git commit, verify_tree.sh	Clean work tree, verified logic.
2. Harvest	Analyze the work for reusable patterns.	ops/lib/skill/skill_lib.sh --harvest	storage/handoff/SKILL_CANDIDATE.md
3. Review	Check the candidate for hallucinations.	(Manual Edit)	A refined, generalized skill definition.
4. Promote	Register the skill in the canon.	ops/lib/skill/skill_lib.sh --promote	New S-LEARN file, updated INDEX.md.
5. Closeout	Finalize the thread.	TASK.md update	Thread completion, ready for merge.

This workflow ensures that "Skill Harvesting" is not an afterthought, but the crowning

achievement of the task.

Works cited

1. Agent Skills :Standard for Smarter AI, accessed February 1, 2026, https://nayakpplaban.medium.com/agent-skills-standard-for-smarter-ai-bde76ea61c13
2. dump-full-main-b818b6193.txt
3. WHAT ARE AGENT SKILLS?, accessed February 1, 2026, https://medium.com/@tahirbalarabe2/what-are-agent-skills-c7793b206daf
4. Agent Skills - OpenAI for developers, accessed February 1, 2026, https://developers.openai.com/codex/skills/
5. Levels of Autonomy for AI Agents - | Knight First Amendment Institute, accessed February 1, 2026, https://knightcolumbia.org/content/levels-of-autonomy-for-ai-agents-1
6. Introducing: Postgres Best Practices - Supabase, accessed February 1, 2026, https://supabase.com/blog/postgres-best-practices-for-ai-agents
7. Rethinking Observability Costs: How Structured Logging Can Save You Thousands, accessed February 1, 2026, https://dev.to/anderson_leite/rethinking-observability-costs-how-structured-logging-can-save-you-thousands-54bh
8. Turning data into a knowledge base - Amazon Bedrock, accessed February 1, 2026, https://docs.aws.amazon.com/bedrock/latest/userguide/kb-how-data.html